# AEP-X Ultra — Instructional Manual

The step-by-step build runbook: exact commands, code, and checks for executing the ADLC Plan

**Companion to** ADLC-Plan, Handoff, Microservices Implementation Guide, Operational Manual & Monetization Strategy

**Prepared** 7 July 2026 · **Updated** 8 July 2026 **Audience** Engineers executing the build

**Scope** Execution-Lock reading (6 services) — since delivered and exceeded, see the addendum below

**Addendum — 8 July 2026: this manual has been executed, and the delivered system goes beyond it.** The repo tree, schemas, and all service scaffolds in Parts 1–4 exist and run — plus the Microservices-Architecture-v2 services (Knowledge, Verification, Cost Optimiser, ML Integration), the full L0–L5 cache, and the SOA Connector Bus with **107 catalogued connectors across 11 category services**. Three defects in this manual's own listings were found and fixed during deployment: the Postgres image must be `pgvector/pgvector:pg16` (not `postgres:16`); Kafka's `CLUSTER_ID` must be a base64 16-byte UUID (the Microservices Guide's example value crashes the broker); and Kafka producers/consumers need reconnect loops — created-once-at-import clients die silently on the compose startup race. For running, verifying, and troubleshooting the *live* system — including UML use case and sequence diagrams of the three load-bearing flows — use the companion **Operational-Manual.pdf**; this document remains the from-scratch build runbook. Delivery status vs. the original commitment is recorded in **Handoff §1a**.

## Contents

## Introduction & Prerequisites

> **Delivered since this manual was written (24 July 2026, `v0.1.0`):** the platform now runs **16 services** (the 6 here + Discovery/Workflow/Safety/Governance + Knowledge/Verification/Cost-Optimiser/ML-Integration + the RFC-0008 `oracle-bridge` and the `marketplace` engine). The **RFC-0008 AI↔blockchain bridge** is live both directions (governed contract read/write; the decision oracle with log-based event subscription), driven from the SDK (`chain`/`oracle` plugins) or the **AEP-X Console** at `:8083`. An opt-in **live-chain mode** (`docker-compose.chain.yml` + `scripts/deploy_contracts.py`) runs the on-chain paths against a local anvil devnet; contracts are compiled and tested on an in-memory EVM in CI. **7 connectors** now have real adapters. See `docs/Console-and-Bridge.md`. The step-by-step build below remains the correct path from zero — the above is what "done" now looks like.

This manual turns the ADLC Plan's phases into literal, ordered steps. It assumes you have accepted the **Handoff** document's committed scope (§2: 6 services, Python SDK, MCP adapter, one vertical, one pilot customer) and resolved its blockers (§5: team size, MCP positioning, pilot contact).

**Read this manual sequentially the first time.** Each Part corresponds to a block of the roadmap and ends with a Definition-of-Done checklist — don't start the next Part until the current one's DoD is met. After the first pass, use the table of contents to jump to whichever Part you're executing.

### Tooling you need before Part 1

| Tool | Version | Check |
| --- | --- | --- |
| Git | 2.40+ | `git --version` |
| Docker Desktop (with Compose v2) | latest | `docker compose version` |
| Python | 3.11+ | `python --version` |
| GitHub CLI (optional but recommended) | latest | `gh --version` |
| jq (for smoke-test scripts) | any | `jq --version` |

WEEKS 1–2

# Part 1 — Governance & Repo Setup

Maps to Handoff Week 1–2 checklist · ADLC Plan §5, §14

## 1.1 Create the GitHub organisation and repository

1. Create the GitHub organisation (e.g. `aepx-core`) and a single monorepo inside it — one repo, not one-per-service, so RFCs/schemas/services/SDK stay in lockstep in Year 1.

```
gh auth login
gh org create aepx-core   # or create via github.com/organizations/new
gh repo create aepx-core/aepx-core --public --description "AEP-X Ultra reference
implementation"
git clone https://github.com/aepx-core/aepx-core.git
cd aepx-core
```

### 1.2 Repository folder structure

2. Create this exact tree. Everything in this manual assumes these paths.

```
aepx-core/
├── governance/
│   ├── constitution.md
│   ├── book-of-laws.md
│   └── decisions/
├── rfcs/
│   ├── RFC-0001-core-protocol.md
│   ├── RFC-0002-identity-trust.md
│   ├── RFC-0003-messaging-workflow.md
│   ├── RFC-0004-discovery-federation.md
│   └── RFC-0005-memory-cache.md
├── schemas/
│   ├── sql/001_init.sql
│   └── openapi/
│       ├── registry.yaml   ├── identity.yaml   ├── trust.yaml
│       ├── memory.yaml     ├── cache.yaml       └── gateway.yaml
├── services/
│   ├── registry/  ├── identity/  ├── trust/
│   ├── memory/    ├── cache/     └── gateway/
├── sdk/python/aepx/
├── cli/aepx_cli/
├── docs/                      (mkdocs site)
├── docker-compose.yml
├── .github/workflows/ci.yml
└── README.md
```

### 1.3 CI skeleton

3 One workflow, matrixed across the 6 services, so every push lints and tests all of them.

.GITHUB/WORKFLOWS/CI.YML

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [registry, identity, trust, memory, cache, gateway]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r services/${{ matrix.service }}/requirements.txt pytest ruff
      - run: ruff check services/${{ matrix.service }}
      - run: pytest services/${{ matrix.service }}/tests -v
```

Verify: push a trivial commit and confirm the Actions tab shows 6 green matrix jobs.

### 1.4 Draft the Constitution and Book of Laws

4 Seed `governance/book-of-laws.md` with the ten constitutional laws already defined in ADLC Plan §2 (Identity Before Interaction … Evolution). Don't re-derive them — copy them in verbatim as the seed, then add one paragraph of rationale per law.

GOVERNANCE/CONSTITUTION.MD — OUTLINE

```
# AEP-X Constitution v1.0 (Draft)

## Preamble
Why AEP-X exists, who it serves, what it will never do.

## Article I — Purpose
## Article II — The Ten Laws            (→ book-of-laws.md)
## Article III — Governance Structure   (interim: Founder + advisor; full TSC at Month 18)
## Article IV — Standards Lifecycle     (Draft → Review → Frozen → Deprecated)
## Article V — Human Authority          (finance, legal, healthcare, child-related: human
final)
## Article VI — Amendment Procedure
```

Target length: 15–30 pages for a Month-1 draft, not the manual's 150–300 page aspiration — this is a working document, not a founding charter yet.

### 1.5 Draft RFC-0001 through RFC-0005

5 Use one template for every RFC. Only these five are in scope for Weeks 1–2 — do **not** start RFC-0006–0010 yet (Handoff §5, blocker 4).

RFCS/_TEMPLATE.MD

```
# RFC-XXXX: <Title>
Status: Draft | Review | Frozen | Deprecated
Author(s):
Created:

## 1. Abstract
## 2. Motivation
## 3. Design Goals
## 4. Specification
## 5. Data Model / Schema           (link to schemas/openapi/*.yaml, schemas/sql/*.sql)
## 6. Security & Compliance Considerations
## 7. Backward Compatibility
## 8. Reference Implementation      (link to services/*)
## 9. Open Questions
```

| RFC | Scope (one line) |
| --- | --- |
| RFC-0001 Core Protocol | Message envelope, message types, capability contract |
| RFC-0002 Identity & Trust | Identity model, 5-component trust formula, 6 trust levels |
| RFC-0003 Messaging & Workflow | Event bus, orchestration modes (defer full Workflow Engine to Part 5) |
| RFC-0004 Discovery & Federation | Capability discovery & ranking (defer federation levels F1+ to post-GA) |
| RFC-0005 Memory & Cache | M0–M6 memory layers, L0–L5 cache layers, TTL policy |

### 1.6 Docs portal skeleton

6

```
pip install mkdocs mkdocs-material
mkdocs new docs
# edit docs/mkdocs.yml: nav: [Home, Governance, RFCs, Services, SDK]
mkdocs build
mkdocs gh-deploy   # publishes to https://aepx-core.github.io/aepx-core/
```

**Part 1 Definition of Done**

- Repo tree matches §1.2 exactly; CI shows 6 green jobs on a dummy PR.
- Constitution + Book of Laws merged to `governance/`, reviewed by at least one person outside the founding team.
- RFC-0001–0005 merged as `Status: Draft` — not more, not fewer.
- Docs site live at a public URL, even if mostly stubs.

WEEK 2

# Part 2 — Data Model & Schemas

Maps to Handoff Week 2 checklist · ADLC Plan §4, §6.1

### 2.1 Canonical data model

1 This is the minimum schema for the 6 in-scope services — not the full canonical model in the manual. Extend it later; don't front-load fields you don't need yet.

SCHEMAS/SQL/001_INIT.SQL

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE agents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organisation_id UUID,
    name TEXT NOT NULL,
    version TEXT NOT NULL DEFAULT '0.0.1',
    trust_score INT DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE capabilities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    version TEXT NOT NULL
);

CREATE TABLE agent_capabilities (
    agent_id UUID REFERENCES agents(id),
    capability_id UUID REFERENCES capabilities(id),
    confidence NUMERIC DEFAULT 0.8,
    cost NUMERIC DEFAULT 0,
    latency_ms INT DEFAULT 0,
    PRIMARY KEY (agent_id, capability_id)
);

CREATE TABLE trust_scores (
    entity_id UUID PRIMARY KEY,
    entity_type TEXT NOT NULL,
    identity_score INT DEFAULT 0,
    behaviour_score INT DEFAULT 0,
    security_score INT DEFAULT 0,
    evidence_score INT DEFAULT 0,
    reputation_score INT DEFAULT 0,
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE memory_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id UUID REFERENCES agents(id),
    layer TEXT NOT NULL CHECK (layer IN ('M1_SESSION','M2_EPISODIC','M3_SEMANTIC')),
    content JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    expires_at TIMESTAMPTZ
);
```

This file is mounted into the Postgres container in Part 3 and runs automatically on first boot.

### 2.2 OpenAPI stubs

2 One `openapi.yaml` per service, generated from the FastAPI app once it exists (Part 3) — for now, hand-write the shape so the SDK/CLI can be built against a contract before the service is done.

```yaml
openapi: 3.0.3
info:
  title: AEP-X Registry
  version: 0.0.1
paths:
  /health:
    get:
      responses: { "200": { description: OK } }
  /agents:
    post:
      requestBody: { required: true }
      responses: { "200": { description: Created } }
    get:
      responses: { "200": { description: List } }
  /agents/{agent_id}:
    get:
      parameters:
        - { name: agent_id, in: path, required: true }
      responses: { "200": { description: OK } }
```

Validate every file: `npx @redocly/cli lint schemas/openapi/*.yaml`

**Part 2 Definition of Done**

- `schemas/sql/001_init.sql` committed and applies cleanly to a fresh Postgres 16 instance.
- 6 OpenAPI files present, all pass `redocly lint`.

WEEKS 3–4

# Part 3 — Core Runtime Scaffold (v0.0.1)

Maps to Handoff Week 3–4 checklist · ADLC Plan §6.1 Wave 1, §6.2 Month 1

### 3.1 docker-compose.yml

1. Postgres + Redis + all 6 services in one file. This is what every developer runs on day one.

   **DOCKER-COMPOSE.YML**

```yaml
version: "3.9"
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: aepx
      POSTGRES_USER: aepx
      POSTGRES_PASSWORD: aepx_dev_only
    ports: ["5432:5432"]
    volumes: ["./schemas/sql:/docker-entrypoint-initdb.d"]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U aepx"], interval: 5s, retries: 5 }

  redis:
    image: redis:7
    ports: ["6379:6379"]
    healthcheck: { test: ["CMD", "redis-cli", "ping"], interval: 5s, retries: 5 }

  identity:
    build: ./services/identity
    ports: ["8001:8000"]
    depends_on: [postgres]

  trust:
    build: ./services/trust
    ports: ["8002:8000"]
    depends_on: [postgres]

  registry:
    build: ./services/registry
    ports: ["8003:8000"]
    depends_on: [postgres]

  memory:
    build: ./services/memory
    ports: ["8004:8000"]
    depends_on: [postgres, redis]

  cache:
    build: ./services/cache
    ports: ["8005:8000"]
    depends_on: [redis]

  gateway:
    build: ./services/gateway
    ports: ["8000:8000"]
    environment:
      - IDENTITY_URL=http://identity:8000
      - TRUST_URL=http://trust:8000
      - REGISTRY_URL=http://registry:8000
      - MEMORY_URL=http://memory:8000
      - CACHE_URL=http://cache:8000
    depends_on: [identity, trust, registry, memory, cache]
```

### 3.2 Service template (used for all 6 services)

2. Every service under `services/*` has the same shape — copy this template and edit `app/main.py` per service.

```
services/<name>/
├── Dockerfile
├── requirements.txt
├── app/
│   ├── main.py
│   └── models.py
└── tests/
    └── test_health.py
```

SERVICES/<NAME>/DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

SERVICES/<NAME>/REQUIREMENTS.TXT

```
fastapi==0.111.0
uvicorn[standard]==0.30.0
pydantic==2.7.1
httpx==0.27.0
```

### 3.3 Registry service

3

SERVICES/REGISTRY/APP/MAIN.PY

```
from fastapi import FastAPI
from pydantic import BaseModel
import uuid

app = FastAPI(title="AEP-X Registry Service", version="0.0.1")

class AgentIn(BaseModel):
    name: str
    organisation_id: str | None = None

class Agent(AgentIn):
    id: str
    version: str = "0.0.1"
    trust_score: int = 0

_AGENTS: dict[str, Agent] = {}

@app.get("/health")
def health():
    return {"status": "ok", "service": "registry"}

@app.post("/agents", response_model=Agent)
def create_agent(payload: AgentIn):
    agent_id = str(uuid.uuid4())
    agent = Agent(id=agent_id, **payload.model_dump())
    _AGENTS[agent_id] = agent
    return agent

@app.get("/agents/{agent_id}", response_model=Agent)
def get_agent(agent_id: str):
    return _AGENTS[agent_id]

@app.get("/agents")
def list_agents():
    return list(_AGENTS.values())
```

**Scaffold, not final:** the in-memory dict is a Week 3 placeholder. Swap it for the `agents` table from Part 2 via SQLAlchemy before the Alpha Gate (Part 4) — flag this explicitly in the PR so it isn't forgotten.

## 3.4 Identity & Trust (minimal stubs)

4

**SERVICES/IDENTITY/APP/MAIN.PY**

```
from fastapi import FastAPI
import jwt, time, uuid

SECRET = "dev-only-change-me"  # replace with env var + Vault before any pilot
app = FastAPI(title="AEP-X Identity Service", version="0.0.1")

@app.get("/health")
def health():
    return {"status": "ok", "service": "identity"}

@app.post("/token")
def issue_token(subject: str):
    payload = {"sub": subject, "iat": int(time.time()), "jti": str(uuid.uuid4())}
    return {"access_token": jwt.encode(payload, SECRET, algorithm="HS256"), "token_type":
"bearer"}
```

**SERVICES/TRUST/APP/MAIN.PY**

```
from fastapi import FastAPI
app = FastAPI(title="AEP-X Trust Service", version="0.0.1")

@app.get("/health")
def health():
    return {"status": "ok", "service": "trust"}

@app.get("/trust/{entity_id}")
def get_trust(entity_id: str):
    # Placeholder value — RFC-0002 5-component formula wired in Wave 3 (Month 5)
    return {"entity_id": entity_id, "trust_score": 50, "level": "Verified"}
```

### 3.5 Memory & Cache

5

SERVICES/MEMORY/APP/MAIN.PY

```
from fastapi import FastAPI
from pydantic import BaseModel
import time

app = FastAPI(title="AEP-X Memory Service", version="0.0.1")
_SESSION_MEMORY: dict[str, list[dict]] = {}

class MemoryWrite(BaseModel):
    agent_id: str
    content: dict

@app.get("/health")
def health():
    return {"status": "ok", "service": "memory"}

@app.post("/memory/session")
def write_session(m: MemoryWrite):
    _SESSION_MEMORY.setdefault(m.agent_id, []).append({**m.content, "ts": time.time()})
    return {"stored": True}

@app.get("/memory/session/{agent_id}")
def read_session(agent_id: str):
    return _SESSION_MEMORY.get(agent_id, [])
```

SERVICES/CACHE/APP/MAIN.PY

```
from fastapi import FastAPI
import redis, os, json

app = FastAPI(title="AEP-X Cache Service", version="0.0.1")
r = redis.Redis(host=os.getenv("REDIS_HOST", "redis"), port=6379, decode_responses=True)

@app.get("/health")
def health():
    return {"status": "ok", "service": "cache", "redis": r.ping()}

@app.get("/cache/{key}")
def get_cache(key: str):
    val = r.get(key)
    return {"key": key, "value": json.loads(val) if val else None, "hit": val is not None}

@app.post("/cache/{key}")
def set_cache(key: str, value: dict, ttl: int = 300):
    r.set(key, json.dumps(value), ex=ttl)
    return {"stored": True, "ttl": ttl}
```

Add `redis==5.0.4` to the cache service's `requirements.txt`.

### 3.6 Gateway

6

SERVICES/GATEWAY/APP/MAIN.PY

```
from fastapi import FastAPI
import httpx, os

app = FastAPI(title="AEP-X Gateway", version="0.0.1")
SERVICES = {
    "identity": os.getenv("IDENTITY_URL", "http://identity:8000"),
    "trust":    os.getenv("TRUST_URL", "http://trust:8000"),
    "registry": os.getenv("REGISTRY_URL", "http://registry:8000"),
    "memory":   os.getenv("MEMORY_URL", "http://memory:8000"),
    "cache":    os.getenv("CACHE_URL", "http://cache:8000"),
}

@app.get("/health")
async def health():
    results = {}
    async with httpx.AsyncClient(timeout=2.0) as client:
        for name, base in SERVICES.items():
            try:
                results[name] = (await client.get(f"{base}/health")).json()
            except Exception as e:
                results[name] = {"status": "down", "error": str(e)}
    return results

@app.post("/execute")
async def execute(agent_id: str, capability: str, payload: dict):
    # v0.0.1: passthrough only. Cache-first routing (RFC-0008 hierarchy) lands in Part 4.
    async with httpx.AsyncClient(timeout=5.0) as client:
        agent = await client.get(f"{SERVICES['registry']}/agents/{agent_id}")
        return {"agent": agent.json(), "capability": capability, "payload": payload}
```

### 3.7 Python SDK v0.0.1

7

```
sdk/python/
├── pyproject.toml
└── aepx/
    ├── __init__.py
    └── agent.py
```

SDK/PYTHON/AEPX/AGENT.PY

```
import httpx

class Agent:
    def __init__(self, name: str, gateway_url="http://localhost:8000",
registry_url="http://localhost:8003"):
        self.name, self.gateway_url, self.registry_url, self.id = name, gateway_url,
registry_url, None

    def register(self):
        r = httpx.post(f"{self.registry_url}/agents", json={"name": self.name})
        r.raise_for_status()
        self.id = r.json()["id"]
        return self.id

    def execute(self, capability: str, **payload):
        r = httpx.post(f"{self.gateway_url}/execute",
                        params={"agent_id": self.id, "capability": capability},
json=payload)
        r.raise_for_status()
        return r.json()
```

```
cd sdk/python
pip install -e .
python -c "from aepx import Agent; a=Agent('tutor'); print(a.register())"
```

## 3.8 CLI

8

CLI/AEPX_CLI/MAIN.PY

```python
import typer
app = typer.Typer()

@app.command()
def init():
    """Scaffold a new AEP-X agent project."""
    typer.echo("Created ./aepx.yaml — edit it, then run `aepx create agent`.")

@app.command()
def create(kind: str):
    """Create a new agent or workflow stub."""
    typer.echo(f"Created {kind} stub in ./agents/{kind}.py")

@app.command()
def run(agent_file: str):
    """Run an agent locally against the Gateway."""
    typer.echo(f"Running {agent_file} against http://localhost:8000 ...")

if __name__ == "__main__":
    app()
```

```
pip install typer
python cli/aepx_cli/main.py init
# time this end to end — target is under 10 minutes, clone to running agent
```

### 3.9 MCP adapter (pulled forward — see Handoff §2)

9. Wrap the Gateway's `/execute` behind an MCP server, exposing each registered capability as an MCP tool. Use the official `mcp` Python SDK's `Server` class:

```
from mcp.server import Server
from mcp.types import Tool
import httpx

server = Server("aepx-gateway")

@server.list_tools()
async def list_tools():
    return [Tool(name="aepx_execute", description="Run an AEP-X agent capability",
                 inputSchema={"type": "object", "properties": {
                     "agent_id": {"type": "string"}, "capability": {"type": "string"}}})]

@server.call_tool()
async def call_tool(name: str, arguments: dict):
    async with httpx.AsyncClient() as client:
        r = await client.post("http://localhost:8000/execute", params=arguments, json={})
        return [{"type": "text", "text": r.text}]
```

This is enough for an MCP client (e.g. Claude Desktop, Claude Code) to call an AEP-X agent as a tool — the concrete proof point for the "memory/cost/evidence layer for MCP agents" positioning.

### 3.10 Tutor Agent demo

10.

```
from aepx import Agent
tutor = Agent("tutor-agent")
tutor.register()
result = tutor.execute("generate_lesson", topic="fractions", grade=5)
print(result)
```

Record a short screen capture of this running end-to-end — it's the artifact you share with the Week 4 pilot contact (Handoff §5, blocker 3).

### 3.11 Smoke test script

11 SCRIPTS/SMOKE-TEST.SH

```
#!/usr/bin/env bash
set -e
echo "Waiting for services..."; sleep 5
curl -sf http://localhost:8000/health | jq .

AGENT_ID=$(curl -sf -X POST http://localhost:8003/agents \
  -H "Content-Type: application/json" -d '{"name":"tutor-agent"}' | jq -r .id)
echo "Created agent $AGENT_ID"

curl -sf -X POST "http://localhost:8000/execute?agent_id=$AGENT_ID&capability=greet" \
  -H "Content-Type: application/json" -d '{}' | jq .

echo "Smoke test passed."
```

```
docker compose up -d --build
chmod +x scripts/smoke-test.sh
./scripts/smoke-test.sh
```

**Part 3 / v0.0.1 Definition of Done**

- `docker compose up` exits clean; `GET /health` on the Gateway reports all 5 backend services `"status": "ok"`.
- Smoke test script passes with no manual intervention.
- Fresh clone → running Tutor Agent demo in **under 10 minutes**, timed and recorded.
- One MCP tool call proxied through the Gateway successfully.
- Git tag `v0.0.1` pushed; CI green on the tagged commit.

MONTHS 2–3

# Part 4 — Alpha Build & Gate Review (v0.1)

Maps to ADLC Plan §6.2 Months 2–3, §7.2 Alpha Gate Review

This Part was originally sketch-level only — Discovery and the Workflow Engine were named at the requirement level (RFC-0003/0004/0005) without runnable code. That gap is now closed: the companion **Microservices Implementation Guide** gives full FastAPI implementations for Discovery, Workflow Engine, and two services this Part doesn't cover at all — Safety Engine and Governance Engine — plus the Kafka/Neo4j infrastructure additions they need. Use §4.1–4.2 below for the build sequence and design rationale; follow the Guide for the actual code.

### 4.1 Discovery Service

1. Add a `/discover` endpoint to a new `services/discovery` service (same template as §3.2). Rank candidates by the formula in ADLC Plan §6.1/RFC-0004: `40% capability match + 25% trust + 15% availability − 10% latency − 10% cost`. Cache ranked results in Redis with a short TTL before falling back to a full registry scan.

   > Full implementation: Microservices Implementation Guide §5.1.

### 4.2 Workflow Engine (minimal)

2. Start with **sequential** orchestration only (defer Parallel/Conditional/Dynamic modes to Beta). A workflow is a JSON list of `{capability, agent_id}` steps; the engine calls the Gateway once per step and threads the output of step N into step N+1's payload.

   > Full implementation (including the `workflow.completed` Kafka event): Microservices Implementation Guide §5.2. That Guide also covers Safety Engine (§5.3) and Governance Engine (§5.4), which sit downstream of the Workflow Engine's events and aren't in this Part at all.

### 4.3 Vector memory (pgvector)

3

```
-- add to schemas/sql/002_vector.sql
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE memory_entries ADD COLUMN embedding vector(1536);
CREATE INDEX ON memory_entries USING hnsw (embedding vector_cosine_ops);
```

Add an embedding call (any provider) in the Memory service before insert; add a `/memory/search` endpoint doing cosine similarity search.

### 4.4 Anti-hallucination validation stub

4

Implement the response contract from ADLC Plan §2 on every agent output: `{answer, confidence, evidence[], verification_status}`. At Alpha, "evidence" can be as simple as "which memory/knowledge record this answer came from" — the full verification pipeline (RFC-0007) is a Beta-phase build, not Alpha.

### 4.5 Running the Alpha Gate Review

5 Five gates, ADLC Plan §7.2. Concrete measurement method for each:

| Gate | How to measure it | Threshold |
|---|---|---|
| 1. Technical | `hey -n 200 -c 10 http://localhost:8000/health` (or `ab`) against Discovery/Memory/Gateway | Discovery <50ms, Memory <20ms, Gateway <100ms, Cache >90% hit on repeat calls |
| 2. Dev Experience | Stopwatch: fresh clone → `docker compose up` → first agent response | Install <5 min, first agent <10 min |
| 3. Cost | Log token counts + cache hit/miss per execution; `cost = tokens_used × model_price`, compare cached vs uncached path | Directional evidence of LLM-call reduction on repeat queries |
| 4. Hallucination | Build a 100-question CSV (`question,expected_answer,source`); run each through the agent; score each response Verified/High/Medium/Unverified per the confidence bands in ADLC Plan §2 | Record actual %, do not assert <1% before this data exists (Critical Evaluation §15.4) |
| 5. Benchmark | Run the same Tutor Agent task through MCP+a bare LLM, LangGraph, and CrewAI; compare latency, cost, lines-of-code, setup time | Publish the comparison table regardless of outcome — this is the credibility engine (Critical Evaluation §15.5 Rec #6) |

**Decision:** proceed to Beta only if Gates 1–2 pass outright and Gates 3–5 have honestly recorded numbers (they do not need to "win" — they need to be true).

**Part 4 / v0.1 Alpha Definition of Done**

- Discovery, Workflow (sequential), vector memory, and the evidence-contract stub are live behind the Gateway.
- All 5 Alpha Gate measurements have been run and recorded (not estimated) in `governance/decisions/alpha-gate-review.md`.
- Go/no-go decision documented and dated.

MONTHS 4–12

# Part 5 — Beta → RC → GA Playbook

Maps to ADLC Plan §6.2, §7.3, §8

From here, the ADLC Plan (§6.2, §7.3) already specifies exit criteria per stage — this Part is the operating cadence for running against them, not new architecture.

### 5.1 Phase kickoff (run at the start of Beta, RC, and GA hardening)

1 **Kickoff agenda template** (30 minutes):

1. Re-state this phase's exit criteria verbatim from ADLC Plan §7.3/§8.
2. Confirm owner per workstream (extend the RACI in Handoff §3).
3. Set the weekly review slot (§5.2) for the duration of the phase.
4. Identify the single biggest risk to hitting the exit criteria — write it down.

### 5.2 Weekly cadence

2 Every week: a 15-minute stand-up against the Command Centre's 5 dashboards (Product/WAU, Technology, Economics, Trust & Safety, Growth — ADLC Plan §11). Record the numbers, don't just discuss them — the point is a time series, not a meeting.

### 5.3 Exit review

3 At the end of each phase, re-run the relevant gate from ADLC Plan §7.3/§8 (Beta exit: 99% reliability, 3 pilots, 100 developers, 10 orgs, 50% cost reduction; RC exit: 99.9% reliability, 90% cache hit, 80% LLM reduction, conformance suite, security audit, 3 production pilots, 1,000 developers). Document actual-vs-target for every line — partial misses are fine, undocumented misses are not.

**Part 5 Definition of Done (per phase)**

- Kickoff agenda run and documented at phase start.
- Weekly dashboard numbers recorded for every week of the phase — no gaps.
- Exit review completed against the ADLC Plan's published criteria before advancing to the next release stage.

MONTH 6 ONWARD

# Part 6 — Operations Runbook

Maps to ADLC Plan §12, OPS-001

## 6.1 Backup & restore

1

```
# Daily full backup
docker exec -t $(docker compose ps -q postgres) pg_dump -U aepx aepx > backup_$(date
+%F).sql

# Restore
cat backup_2026-07-14.sql | docker exec -i $(docker compose ps -q postgres) psql -U aepx
aepx
```

Automate the backup command via a scheduled GitHub Action or cron once off a laptop and onto real infrastructure; retain 35 days per ADLC Plan §12.

## 6.2 Incident response (P1–P4)

2

| Severity | Example | Response time | First action |
|---|---|---|---|
| P1 Critical | Gateway down, data loss | 15 minutes | Page the on-call owner; open an incident doc from the template (Part 7) |
| P2 High | One core service down, others up | 1 hour | Restart via `docker compose restart <service>`, check logs |
| P3 Medium | Degraded latency, no outage | 4 hours | Log to backlog, monitor |
| P4 Low | Minor defect | Next release | File as a normal issue |

## 6.3 Disaster recovery test

3 Quarterly: restore the latest backup into a throwaway environment and run the Part 3 smoke test against it. If the smoke test doesn't pass on the restored copy, the backup strategy is broken — fix it before the next quarter, not after an actual incident.

REFERENCE

# Part 7 — Templates & Appendices

## 7.1 ADR (Architecture Decision Record) template

```
# ADR-XXXX: <Decision title>
Status: Proposed | Accepted | Superseded
Date:

## Context
## Decision
## Consequences (what gets easier, what gets harder)
## Alternatives considered
```

## 7.2 Service README template

```
# <Service Name>
One-line purpose.

## Run locally
docker compose up <service>

## Endpoints
- GET /health
- ...

## Definition of Done for this service
- [ ] OpenAPI contract published
- [ ] Unit tests >85% coverage
- [ ] Audit events emitted
- [ ] Docs page published
```

## 7.3 Incident report template

```
# Incident: <short title>
Severity: P1-P4      Detected:      Resolved:

## Summary
## Timeline
## Root cause
## Impact
## Remediation
## Follow-up actions (owner + due date)
```

### 7.4 Weekly Command Centre report template

```
# Week of <date>
Product:     WAU ___   MAU ___   New agents created ___
Technology: Availability ___%  Gateway p95 ___ms  Cache hit ___%
Economics:  Cost/execution ___  LLM calls avoided ___%
Trust&Safety: Evidence coverage ___%  Incidents ___
Growth:     Developers ___  Orgs ___  Pilot conversations ___
Biggest risk this week:
Decision needed from leadership:
```

## Troubleshooting & FAQ

| Symptom | Likely cause | Fix |
|---|---|---|
| `docker compose up` hangs on Postgres | Init SQL script has an error | Check `docker compose logs postgres`; fix `001_init.sql`, then `docker compose down -v` to wipe the volume and retry |
| Gateway `/health` shows a service as `"down"` | Service container crashed on boot, or wrong URL env var | `docker compose logs <service>`; confirm the env var in `docker-compose.yml` matches the service name |
| Cache service can't reach Redis | Missing `redis` package, or wrong host | Confirm `redis==5.0.4` in requirements.txt and `REDIS_HOST=redis` (the Compose service name, not `localhost`) |
| SDK `Agent.register()` raises a connection error | Registry not up yet, or wrong port | Confirm `docker compose ps` shows Registry healthy on 8003 before running the SDK |
| CI fails only in the matrix, not locally | Missing dependency pin, or a service-specific env var not set in CI | Pin exact versions in `requirements.txt`; add any needed env vars to the workflow step |